KU LEUVEN

FACULTY ENGINEERING TECHNOLOGY

CAMPUS GROEPT

Imec exercise

8/11/2026



Matas REMEIKA

1 INTRODUCTION

In modern LLM-based operational knowledge assistants, input payloads are dynamically constructed from multiple data streams - primarily system instructions, retrieved knowledge base documents, and user queries. As usage scales, this multi-component payload consistently drives up input token counts, leading to increased API expenditure, higher prefill latency, and potential context-window saturation.

This report investigates token-optimization techniques designed to heavily reduce this operational overhead without compromising response accuracy. Specifically, we examine two targeted input-reduction strategies: **Context Pruning** (utilizing Levenshtein distance to isolate only the most relevant sentences from retrieved documents) and **System Prompt Compression** (filtering out low-information lexical bloat from static instructions). Finally, we present a lightweight, Python-based proof of concept (PoC) to validate these strategies, demonstrating quantifiable improvements in simulated processing time and API costs.

2 OPTIMIZATION TECHNIQUES

2.1 Context Pruning & Dynamic Threshold Filtering

- How it works: In standard RAG, a retrieval engine fetches multiple long documents and passes them entirely to the LLM. Context pruning splits those documents into sentences and computes a relevance score against the user's query using lexical or semantic similarity metrics. A dynamic threshold filter then rejects the passages falling below a specific score threshold and retains only the top-K most relevant chunks.
- Tokens reduced: Input tokens
- Expected benefits: Reduces context length, improves answer reliability, by eliminating irrelevant noise, lowers API costs.
- Possible disadvantges: Filtering thresholds risk pruning low score chuncks that contain subtle and critical facts required to fully answer user's query [1] .

2.2 System prompt & Instruction Compression

- How it works: Prompt compression algorithms analyze the information that comes together with every single API request and strip out low-information filler words without violating instructions indicated in the API request.
- Tokens reduced: Input tokens
- Expected benefits: Reduction in prompt sizes yield massive operational savings
- Possible disadvantages: Over-compressing system prompts can strip away formatting schemas, causing the LLM to behave unexpectedly [2] .

2.3 Conversation History Sliding Window & Summarization

- How it works: As a chat session progresses, the accumulated conversation history is appended to every new prompt, making input token usage grow rapidly over time. A sliding window mechanism solves this by retaining the last N turns, while periodic summarization collapses older dialogue turns into a concise summary block.
- Tokens reduced: Input tokens.
- Expected benefits: Keeps per-request input costs flat over extended chat sessions and prevents performance or latency degradation as the conversation grows.
- Possible disadvantages: Poor summarization can cause model to lose essential instructions, constraints, or specific user preferences mentioned earlier in the chat [3] .

2.4 Response Length Bounding & Structured Output Constraints

- How it works: LLM outputs are unrestricted by default, which often leads to long explanations. This technique uses generation parameters and enforced structured output schemas (for example JSON format) to constrain the length and format of the response.
- Tokens reduced: Output tokens
- Expected benefits: Lowers billing expenses, reduces user-perceived latency.
- Possible disadvantages: Responses can become overly rigid and lack necessary explanatory context required for complex operational troubleshooting queries [4] .

3 PREFERRED APPROACH & PROTOTYPE DESIGN

To optimize token usage, this proof of concept (PoC) combines Context pruning and System Prompt Compression.

3.1 Rationale For Selection

- Context pruning: Rather than passing entire retrieved documents to the LLM, the system isolates the most relevant information. I selected the Levenshtein distance algorithm because it provides a lightweight, transparent method to calculate a lexical similarity score between the user’s query and each sentence in the context, allowing the system to extract only the top-k most relevant sentences.
- System prompt compression: To reduce the static overhead of system instructions, I implemented a lexical filter that identifies and removes low-information "fluff" words that do not alter the core directive of the prompt.

3.2 Implementation & Evaluation Strategy

The PoC is implemented via a Python script (main.py). The execution flow operates in two stages:

1. Pruning: The script chunks the raw context into individual sentences, calculates the similarity score of each against the user query, and retains only the top two highest-scoring sentences.
2. Compression: The script loads a predefined dictionary of unnecessary "fluff" words and scans the system prompt, stripping these words out to reduce token footprint.

To evaluate the success of the optimized prompt, the script compares the total character lengths of the baseline payload versus the optimized payload. Furthermore, the script applies an assumed processing speed and an arbitrary cost (e.g., $0.50 per 1 million characters) to quantify the simulated reduction in Time-to-First-Token (TTFT) latency and financial API costs.

3.3 Determining Acceptable Quality Trade-offs

While the implemented algorithms significantly reduce the character count in the payload, token reduction is only valuable if it does not degrade the LLM's performance. Future evaluation requires running a golden dataset of test queries through both the baseline and optimized pipelines to compare answer accuracy. If the token volume is heavily reduced with no significant drop in response accuracy, the trade-off is highly successful.

3.4 Drawbacks & Proposed Improvements

Reflecting on the current prototype design, there are two notable limitations that would need to be addressed in a production environment:

- Static Sentence Limits: The current pruning algorithm hardcodes the extraction to the top 2 sentences. For queries requiring synthesized answers across long contexts, this is too restrictive. A better approach would be dynamic thresholding: identifying the highest-scoring sentence, and retaining any other sentences whose scores fall within a specific percentage margin of that top score.
- Context-Dependent Fluff Words: The static deletion mechanism assumes certain English words are universally "low information," which is not always true depending on context. A robust alternative would be utilizing a smaller, highly efficient LLM specifically tasked with compressing the prompt. However, this introduces a new trade-off: we would need to measure whether the tokens saved on the primary LLM outweigh the tokens spent querying the smaller compression LLM.

4 BIBLIOGRAPHY

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020. Available: arXiv:2005.11401

[2] H. Jiang, L. Wu, C. Zhang, Q. Yang, and J. Zhou, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. Available: Microsoft Research Publications

[3] R. Anil et al., "PaLM 2 Technical Report," *Google Research*, 2023. Available: arXiv:2305.10403

[4] OpenAI Developer Documentation, "Structured Outputs and Token Management Guide," OpenAI API Documentation. Available: OpenAI Guides